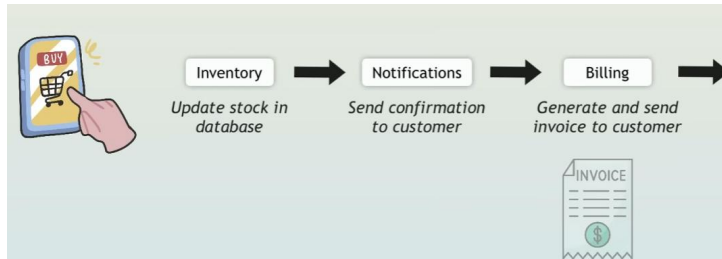
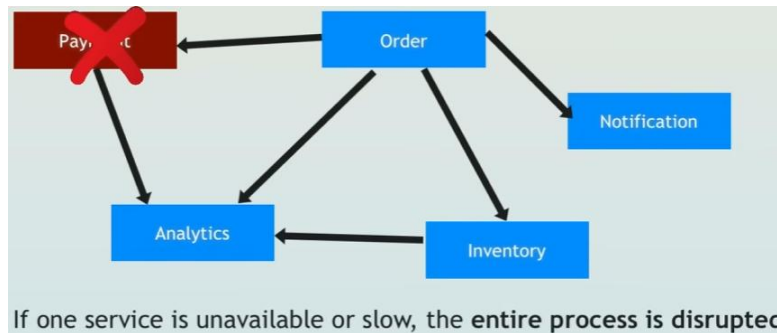


➤ Need of Kafka:

- ⌘ Lets say you have a microservice architecture where one service is calling other directly depending upon their needs.
- ⌘ For example, lets say one order service
order placed -> inventory update -> customer confirmation mail -> etc etc

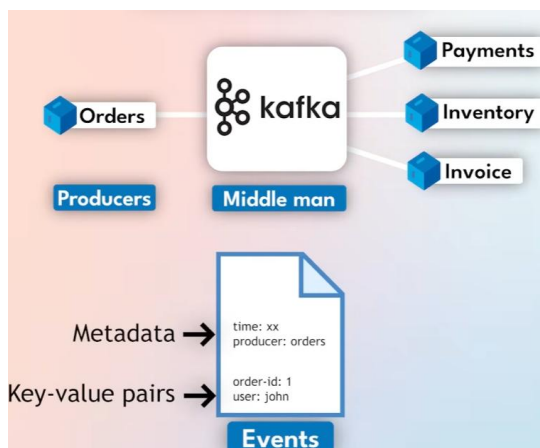


- ⌘ In this process, if the requests are too many and if one service gets stuck, then whole requests will be hampered.
- ⌘ Here every services are **tightly coupled**.



- ⌘ This is synchronous calls, what we want is asynchronous. Services should not interact with each other directly.

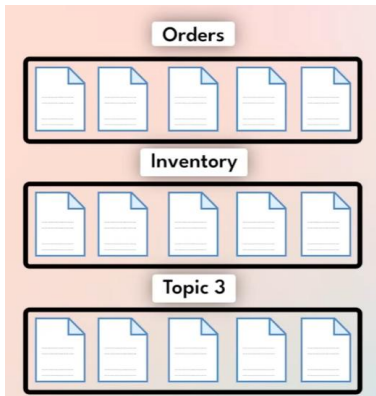
This is where message queue comes into picture.



- ⌘ The services that create the events are called **producers**.

Where do the events get saved?

- In Kafka, the events are ordered logically, which are called topics.



- ⌘ Here Orders topic, Inventory topic, Topic 3 .. are there.
- ⌘ NOTE: Topics are not physical storage, these are just there to separate the event types logically.

You can decide which topics has to be created based on your need.

```
// Define the topics
NewTopic ordersTopic = new NewTopic("orders", 3, (short) 1);
NewTopic inventoryTopic = new NewTopic("inventory", 2, (short) 1);
NewTopic notificationsTopic = new NewTopic("notifications", 4, (short) 1);

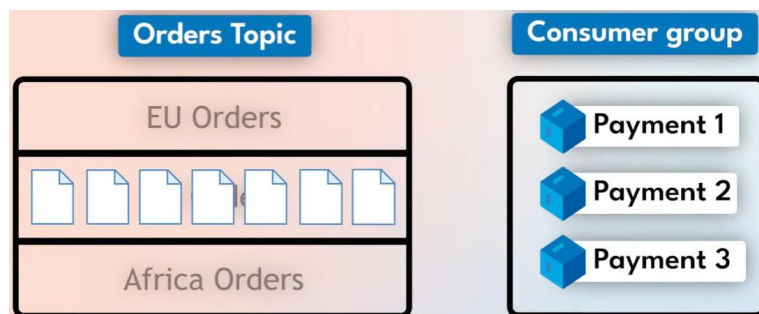
// Add topics to a list
List<NewTopic> topics = Arrays.asList(ordersTopic, inventoryTopic, notificationsTopic);

// Create the topics
adminClient.createTopics(topics)
    .all() // Ensures the operation completes
    .get();
```

- ⌘ The Producer service will create event and handle it to Kafka. Because it trusts that Kafka will deliver the event to the right consumer(s).
- Consumers can subscribe to any topic.
 - ⌘ When an event (consumer's subscribed type) is generated by the producers, the consumer will get notified about that.
 - ⌘ For example lets say one order has been created by customer
 - ⌘ Orders service will create an event of Orders topic type.
 - ⌘ Notification service will be notified on that and send notifications.
 - ⌘ Inventory topic will also be notified and might create an Inventory topic event. And ..etc etc.



- ✧ Chain of events can be triggered.
 - ✧ For example, one event is triggered and consumed by a service.
 - ✧ Now that service again created one event and so on.
- Now, as the events are logically separated with Topics, but it needs to be more specific.
 - ✧ For example lets say US Orders, India Orders, Africa Orders ..like this.
 - ✧ Now lets say, some consumers are there like US Order consumer, India Order consumers etc who will handled those partitions.
 - ✧ So, when you create multiple consumers for a particular partition, it is called **Consumer Group**.
 - ✧ In technical term, when you create multiple instances of a specific micro-service then it is called Consumer Group.



- ✧ Kafka will distribute the load to the consumers of a consumer group automatically.
- At the end, the data is persisted (stored) in Kafka servers called **Brokers** as long as it is needed.
 - ✧ This is the difference between Kafka and other message queues.
 - ✧ In Kafka, the data are not disappeared after it is consumed by the consumers, it is persisted in the disk for future use or analytic purposes.
- ✧
- F
- F

➤ In Simple Terms:

- ↪ The events are **logically** divided into **Topics**. (Order, Inventory, ...etc)
- ↪ The Topics are further divided into **Partitions**. (Ind Order, EU Order ...etc)
(I am dividing the topic by country)
- ↪ Each partition can contain multiple messages like
order1, order2 ...etc

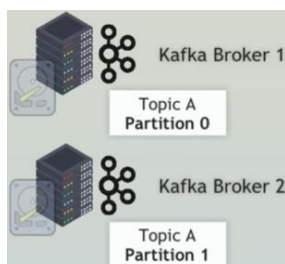
```
Partition 0 (India):  
offset 0 → order_101 (India)  
offset 1 → order_102 (India)  
offset 2 → order_103 (India)  
offset 3 → order_104 (India)  
...  
offset N → order_999999 (India)
```

- ↪ Each message inside the *Partition* will be having one unique offset.
- ↪ Each time one order is placed in India, one message will be added to this India's Order partition list.
- ↪ **Consumer Group** is defined at **Topic** level (not Partition level).
 - ↪ The whole Consumer Group will subscribe to a Topic.
 - ↪ Its Kafka's duty to assign the partition (of Consumer group's subscribed Topic) to the consumer present inside that Consumer Group.
 - ↪ **One partition can be assigned to only one Consumer.**
 - * Lets say you have 3 partitions in a topic, and 5 consumers inside the consumer group, then 2 consumers will always be idle.

```
Consumer 1 → Partition 0  
Consumer 2 → Partition 1  
Consumer 3 → Partition 2  
Consumer 4 → idle  
Consumer 5 → idle
```

* **Max parallelism = number of partitions**

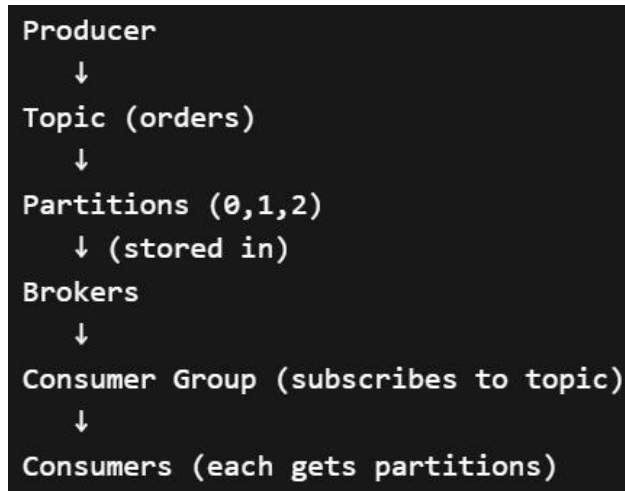
- ↪ Fewer groups + more partitions → better parallelism and performance
- ↪ Kafka Broker is the actual system that **stores & servers Kafka data**.



- ↪ It handles *produce requests (write data)*, *consumer requests (read data)*.
- ↪ One broker can contain multiple Partitions (from same or different topics)

Broker = physical storage + processing unit for partitions

Topic = logical stream of data, split into partitions for scalability



-  **F**
 **F**
 **F**
 **F**
 **F**
 **F**
 **F**
 **F**
 **F**
 **F**
 **F**
 **F**
 **F**